

Behavioral Patterns – Part 2

Observer · State · Strategy · Template Method

Sub-Chapter 02·06

Observer	Notify all subscribers when something changes
State	Object changes behaviour when its state changes
Strategy	Swap algorithms at runtime without changing the caller
Template Method	Define skeleton of algorithm; subclasses fill in the steps

1 Observer Pattern

Notify all subscribers when something changes

What is it?

The Observer pattern defines a **one-to-many** dependency between objects. When one object (the Subject) changes its state, all dependents (Observers) are notified and updated automatically. Think of it like a YouTube subscription: when a channel uploads a video, every subscriber gets a notification.

Real-world example: YouTube Channel & Subscribers

A **YouTubeChannel** keeps a list of **Subscriber** objects. When `uploadVideo()` is called it loops through all subscribers and calls `update()` on each one.

```
import java.util.ArrayList;
import java.util.List;

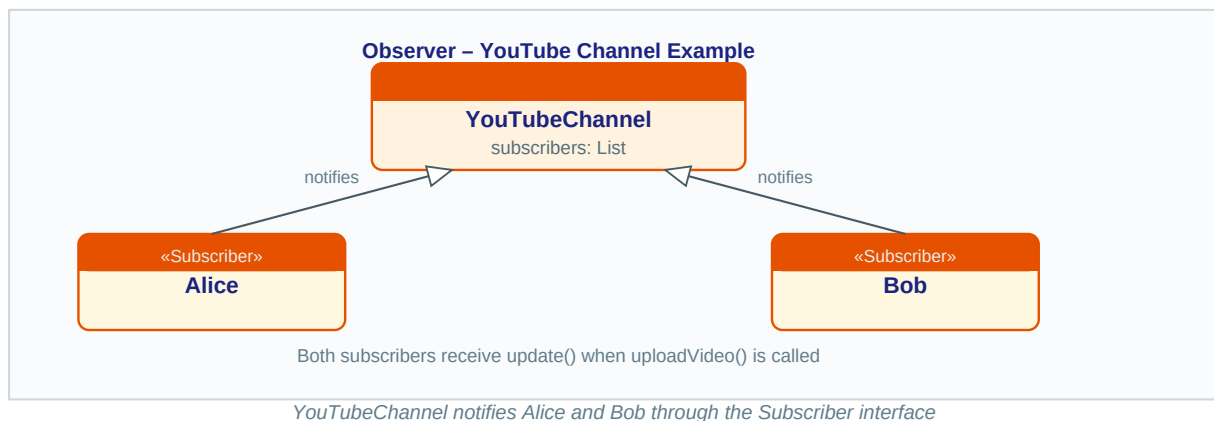
interface Subscriber { void update(String video); }

class YouTubeChannel {
    private List<Subscriber> subscribers = new ArrayList<>();
    private String name;
    YouTubeChannel(String name) { this.name = name; }
    void subscribe(Subscriber s) { subscribers.add(s); }
    void uploadVideo(String title) {
        System.out.println(name + " uploaded: " + title);
        for (Subscriber s : subscribers) s.update(title);
    }
}

class User implements Subscriber {
    private String name;
    User(String name) { this.name = name; }
    public void update(String video) {
        System.out.println("  " + name + " got notified: " + video);
    }
}

// Usage
YouTubeChannel channel = new YouTubeChannel("TechTalks");
channel.subscribe(new User("Alice"));
channel.subscribe(new User("Bob"));
channel.uploadVideo("Design Patterns 101");
// TechTalks uploaded: Design Patterns 101
//  Alice got notified: Design Patterns 101
//  Bob got notified: Design Patterns 101
```

Structure Diagram



Key Rules

- The subject (YouTubeChannel) holds a list of observers (Subscriber).
- Observers register themselves via `subscribe()` / `unsubscribe()`.
- The subject calls `update()` on every observer — it does not care about their concrete type.
- Observers are loosely coupled: you can add or remove them at runtime.

Q1. When should I use Observer?

Use it when a change in one object should automatically trigger updates in many other objects, without tight coupling. Classic cases: UI event listeners, MVC (Model notifies View), pub/sub messaging, Spring `ApplicationEventPublisher`.

Q2. What is the difference between Observer and EventListener in Java?

`EventListener` is Java's built-in application of the Observer pattern. The pattern describes the concept; `EventListener` / `ActionListener` are concrete implementations of it.

Q3. Can the Subject push data, or do Observers pull it?

Both are valid. Push: subject sends data in `update()`. Pull: subject sends a reference to itself and observers call getter methods. Push is simpler; pull gives observers more control.

Q4. What is a real Java framework example?

`java.beans.PropertyChangeListener`, Spring `ApplicationEvent` / `@EventListener`, and RxJava `Observable` all implement the Observer pattern.

Q5. What goes wrong if you forget to unsubscribe observers?

Memory leaks! The subject keeps a reference to observers, preventing garbage collection. Always remove observers when they are no longer needed.

2 State Pattern

Object changes behaviour when its state changes

What is it?

The State pattern lets an object change its behaviour when its internal state changes. Instead of a big if/switch block checking the current state, each state is its own class that handles behaviour and triggers the next transition.

Real-world example: Traffic Light

A **TrafficLight** delegates behaviour to the current **LightState**. Each concrete state class (**RedState**, **GreenState**, **YellowState**) prints its message and sets the next state.

```
interface LightState { void handle(TrafficLight light); }

class TrafficLight {
    private LightState state;
    TrafficLight() { state = new RedState(); }
    void setState(LightState s) { state = s; }
    void change() { state.handle(this); }
}

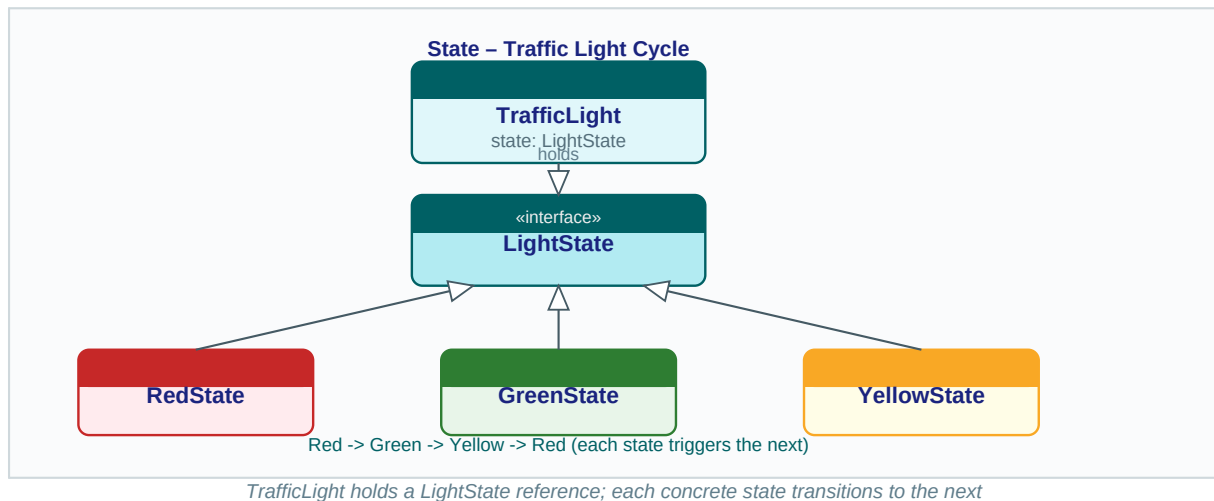
class RedState implements LightState {
    public void handle(TrafficLight light) {
        System.out.println("RED - Stop!");
        light.setState(new GreenState());
    }
}

class GreenState implements LightState {
    public void handle(TrafficLight light) {
        System.out.println("GREEN - Go!");
        light.setState(new YellowState());
    }
}

class YellowState implements LightState {
    public void handle(TrafficLight light) {
        System.out.println("YELLOW - Slow down!");
        light.setState(new RedState());
    }
}

// Usage
TrafficLight light = new TrafficLight();
light.change(); // RED - Stop!
light.change(); // GREEN - Go!
light.change(); // YELLOW - Slow down!
light.change(); // RED - Stop!
```

Structure Diagram



Key Rules

- The context (TrafficLight) delegates state-specific behaviour to the current State object.
- Each concrete state handles the request and decides the next state.
- Adding a new state means adding a new class — no existing code changes.
- Eliminates large if/switch chains that check the current state type.

Q1. How does State differ from Strategy?

In Strategy, the algorithm is chosen by the client and does not change itself. In State, the object transitions between states automatically based on its own logic.

Q2. What if the state needs to access the context's data?

Pass the context (TrafficLight) as a parameter to handle(). The state can then call getters on the context to read or change data.

Q3. Can states transition to themselves (self-loops)?

Yes. A state can call `light.setState(new RedState())` even from within RedState, effectively resetting to the same state.

Q4. Name a real Java / Android example.

TCP connection states (CLOSED, LISTENING, ESTABLISHED), Android Activity lifecycle, and vending machine logic are classic State pattern examples.

Q5. What is the difference between State and an FSM (Finite State Machine)?

The State pattern is the OOP implementation of an FSM. Each class is a node and `setState()` calls are the transitions.

3 Strategy Pattern

Swap algorithms at runtime without changing the caller

What is it?

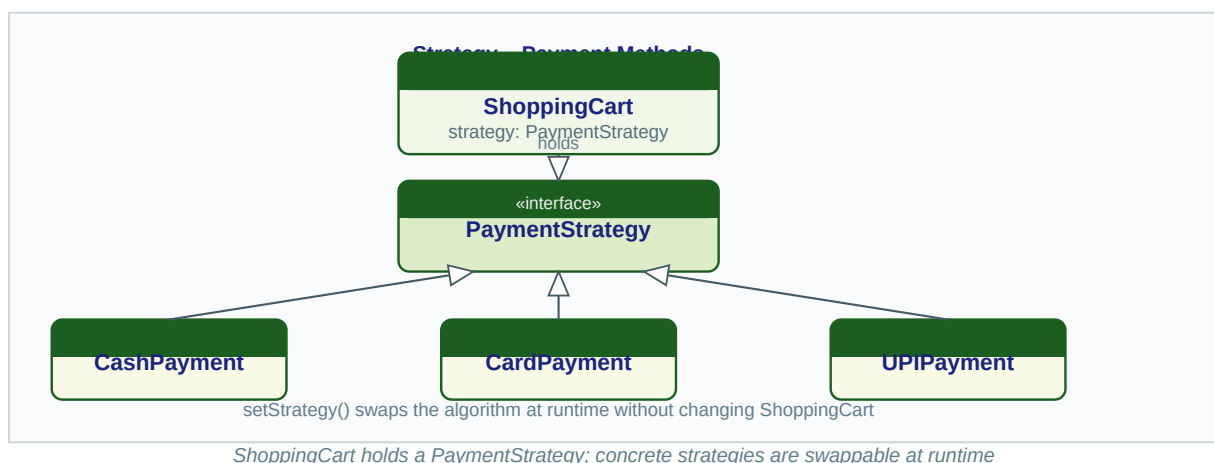
The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. The caller (context) does not know which algorithm is running — it just calls a common interface. You can swap the algorithm at runtime.

Real-world example: Payment at Checkout

A **ShoppingCart** uses a **PaymentStrategy**. The client sets the strategy to **CashPayment**, **CardPayment**, or **UPIPayment** before calling `checkout()`. The cart code never changes.

```
interface PaymentStrategy { void pay(int amount); }
class CashPayment implements PaymentStrategy {
    public void pay(int amount) { System.out.println("Paid Rs " + amount + " in Cash"); }
}
class CardPayment implements PaymentStrategy {
    public void pay(int amount) { System.out.println("Paid Rs " + amount + " by Card"); }
}
class UPIPayment implements PaymentStrategy {
    public void pay(int amount) { System.out.println("Paid Rs " + amount + " via UPI"); }
}
class ShoppingCart {
    private PaymentStrategy strategy;
    void setStrategy(PaymentStrategy s) { strategy = s; }
    void checkout(int amount) { strategy.pay(amount); }
}
// Usage
ShoppingCart cart = new ShoppingCart();
cart.setStrategy(new CashPayment());
cart.checkout(500); // Paid Rs 500 in Cash
cart.setStrategy(new UPIPayment());
cart.checkout(1200); // Paid Rs 1200 via UPI
```

Structure Diagram



Key Rules

- Define a common interface (**PaymentStrategy**) for all algorithm variants.
- The context (**ShoppingCart**) holds a reference to the interface, not a concrete class.
- Use `setStrategy()` to inject or swap algorithms at runtime.
- Each strategy is independently testable and replaceable without touching the context.

Q1. How is Strategy different from a simple if/else switch on payment type?

With if/else you must edit ShoppingCart every time you add a new payment type. With Strategy you just add a new class — the cart code is untouched (Open/Closed Principle).

Q2. Can I use a lambda instead of a full class in Java 8+?

Yes. Since PaymentStrategy is a functional interface (one method) you can write:

`cart.setStrategy(amount -> System.out.println('Paid ' + amount + ' via UPI'))`; This is the most concise form.

Q3. What is the difference between Strategy and Template Method?

Template Method uses inheritance — subclasses override specific steps of a fixed skeleton. Strategy uses composition — the whole algorithm is swapped via an interface reference.

Q4. Name a real Java library example.

`java.util.Comparator` is a Strategy. `Collections.sort(list, comparator)` lets you pass any sorting algorithm at runtime without changing the sort method itself.

Q5. What happens if no strategy is set before calling checkout()?

You get a `NullPointerException`. Guard against this with a default strategy or a null check inside `checkout()`.

4 Template Method Pattern

Define skeleton of algorithm; subclasses fill in the steps

What is it?

The Template Method pattern defines the **skeleton of an algorithm** in a base class, deferring some steps to subclasses. The overall structure stays the same; subclasses override only the parts that differ. The template method is usually marked final so subclasses cannot change the order of steps.

Real-world example: Making a Hot Drink

Both Tea and Coffee follow the same steps: **boilWater** -> **brew** -> **pourInCup** -> **addCondiments**. Only brew() and addCondiments() differ, so those are declared abstract and overridden by subclasses.

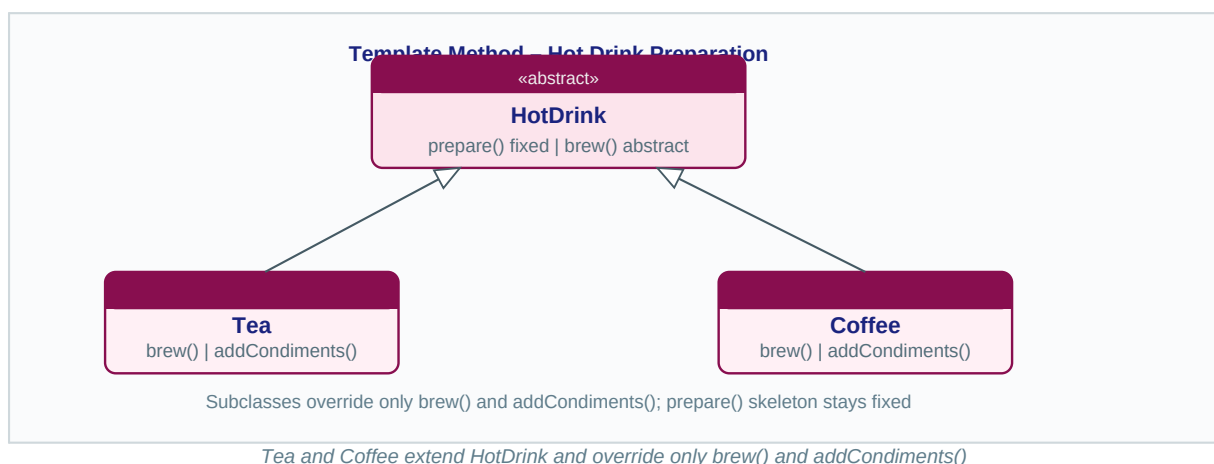
```
abstract class HotDrink {
    // Template method -- defines the skeleton
    final void prepare() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }
    void boilWater() { System.out.println("Boiling water"); }
    void pourInCup() { System.out.println("Pouring into cup"); }
    abstract void brew();
    abstract void addCondiments();
}

class Tea extends HotDrink {
    public void brew() { System.out.println("Steeping the tea"); }
    public void addCondiments() { System.out.println("Adding lemon"); }
}

class Coffee extends HotDrink {
    public void brew() { System.out.println("Dripping coffee"); }
    public void addCondiments() { System.out.println("Adding sugar and milk"); }
}

// Usage
System.out.println("--- Making Tea ---");
new Tea().prepare();
System.out.println("--- Making Coffee ---");
new Coffee().prepare();
```

Structure Diagram



Key Rules

- The template method (prepare()) is declared final — subclasses cannot reorder the steps.
- Common steps (boilWater, pourInCup) are implemented in the abstract class.
- Variable steps (brew, addCondiments) are declared abstract — subclasses must override them.
- Hollywood Principle: the base class calls the subclass methods, not the other way around.

Q1. What is the Hollywood Principle?

It means 'Don't call us, we'll call you.' The abstract base class calls the subclass's methods (brew, addCondiments). The subclass never calls the parent's prepare() directly. This inverts the usual control flow.

Q2. What are hook methods?

Hooks are optional steps with a default (often empty) implementation in the base class. Subclasses may override hooks to add behaviour at specific points without being forced to.

Q3. How is Template Method different from Strategy?

Template Method uses class-level inheritance — decided at compile time. Strategy uses object-level composition — the algorithm can be changed at runtime.

Q4. Name a real Java framework example.

javax.servlet.HttpServlet: doGet() and doPost() are hooks you override. The servlet container calls service() (the template method) which dispatches to them.

Q5. What is the risk of overusing Template Method?

Deep inheritance hierarchies become fragile. If the base class adds a new abstract method, every subclass must implement it. Prefer composition (Strategy) when variation is high.



Quick Reference Cheat Sheet

All four patterns at a glance

Pattern	Intent	Key method	Real Java example	Spot violation
Observer	Notify many objects when one changes	update() / notifyObservers()	java.beans.PropertyChangeListener, Spring @EventListener	Subject directly calls concrete classes instead of the Observer interface
State	Change behaviour when internal state changes	handle(Context ctx)	TCP connection states, Android Activity lifecycle	Giant if/switch in the context checking the current state type
Strategy	Swap algorithm families at runtime	execute() / pay() / sort()	java.util.Comparator, Collections.sort()	Context instantiates concrete strategies with 'new' instead of receiving them
Template Method	Fix algorithm skeleton; let subclasses fill steps	templateMethod() marked final	HttpServlet.service() -> doGet()/doPost()	Template method is not final, allowing subclasses to change the order

Side-by-side Comparison

Aspect	Observer	State	Strategy	Template Method
Relationship	Subject notifies Observers	Context delegates to State	Context holds Strategy	Base class calls subclass hooks
Mechanism	Composition + interface	Composition + interface	Composition + interface	Inheritance (abstract class)
Swappable at runtime?	Yes (subscribe/unsubscribe)	Yes (setState)	Yes (setStrategy)	No (fixed at compile time)
Who decides change?	Caller calls notify()	State decides next state	Client calls setStrategy()	Base class calls abstract hooks
Main benefit	Decoupled event handling	Eliminates if/switch on state	Open/Closed: add strategies freely	Reuse common steps, vary the rest